

Homework 6

- **Submit your solutions electronically on the course Gradescope site as PDF files.** If you plan to typeset your solutions, please use the $\text{\LaTeX}$ solution template on the course web site. If you must submit scanned handwritten solutions, please use a black pen on blank white paper and a high-quality scanner app (or an actual scanner, not just a phone camera). We will mark difficult to read solutions as incorrect and move on.
- **Every homework problem must be done *individually*.** Each problem needs to be submitted to Gradescope before midnight of the due date which can be found on the course website: <https://ecealgo.com/su26/homeworks.html>.
- For nearly every problem, **we have covered all the requisite knowledge required to complete a homework assignment prior to the “assigned” date.** This means that there is no reason not to begin a homework assignment as soon as it is assigned. Starting a problem the night before it is due a recipe for failure.

Policies to keep in mind

- **You may use any source at your disposal**—paper, electronic, or human—but you *must* cite *every* source that you use, and you *must* write everything yourself in your own words. See the academic integrity policies on the course web site for more details.
- **Being able to clearly and concisely explain your solution is a part of the grade you will receive.** Before submitting a solution ask yourself, if you were reading the solution without having seen it before, would you be able to understand it within two minutes? If not, you need to edit. Images and flow-charts are very useful for concisely explain difficult concepts.

See the course web site (<https://ecealgo.com/su26>) for more information.

If you have any questions about these policies,
please don't hesitate to ask in class, in office hours, or on Piazza.

Extra Instructions Solutions to a dynamic programming problem have (at minimum) three things:

- A recurrence relation
- A *brief* description of what your recurrence function represents and what each case represents.
- A *brief* description of the memory element/storage and how it's filled in.

1. Graphs are a powerful tool to model many phenomena. The edges of a graph model pairwise relationships. It is natural to consider higher order relationships. Indeed *hypergraphs* provide one such modeling tool. A hypergraph $G = (V, \mathcal{E})$ consists of a finite set of nodes/vertices V and a finite set of hyper-edges $\mathcal{E}$. A hyperedge $e \in \mathcal{E}$ is simply a subset of nodes and the cardinality of the subset can be larger than two. An undirected graph is a hypergraph where each hyper-edge is of size exactly two.

Here is an example: $V = \{1, 2, 3, 4, 5\}$, $\mathcal{E} = \{\{1, 2\}, \{2, 3, 4\}, \{1, 3, 4, 5\}, \{2, 5\}\}$. The representation size of a hypergraph is $|V| + \sum_{e \in \mathcal{E}} |e|$.

An alternating sequence of nodes and edges $x_1, e_1, x_2, e_2, \dots, e_{k-1}, x_k$ where $x_i \in V$ for $1 \leq i \leq k$ and $e_j \in \mathcal{E}$ for $1 \leq j \leq k-1$ is called a path from u to v if: (i) $x_1 = u$, $x_k = v$, and (ii) for all $1 \leq j < k$, $x_j \in e_j$ and $x_{j+1} \in e_j$.

Given a hypergraph $G = (V, \mathcal{E})$ and two nodes $u, v \in V$, we say that u is connected to v if there is a path from u to v . We say that a hypergraph is connected if each pair of nodes u, v in G are connected. Describe an algorithm that, given a hypergraph G , checks whether G is connected in linear time. In essence, describe a reduction of this problem to the standard graph connectivity problem.

2. **BFS Tree Explosion with Degree Bound** Let $G = (V, E)$ be a directed graph with n vertices, and suppose every vertex has out-degree at most m . Fix a start vertex s , and define a *BFS tree* as the parent-child tree returned by BFS starting at s , where the queue resolves neighbor order arbitrarily (i.e., not sorted). Two BFS trees are considered *distinct* if their parent-child structure differs. What is a tight upper bound on the maximum number of distinct BFS trees that can be produced from a single fixed start vertex s , as a function of n and m ?
3. Assume we have a graph G with n vertices and m equal weight edges. We have two balls B_1, B_2 that will move along their designed path P_1, P_2 , where P_1 has length N_1 and P_2 has length N_2 . Both balls will start at $P_1[1]$ and $P_2[1]$, and can move only forward along their designed paths. To be more precise, we define a valid move for a ball is either stay in it's current node, or move to the next node on its path. Each ball makes exactly one legal move in each round. A sequence, specifying in each round a valid move for each robot, is a plan.

The score of a plan is the maximum distance of the two balls from each other at any moment during the execution of the plan.

Describe an algorithm that computes the minimum score plan for the two balls. Your algorithm should be as fast as possible.